

6. Tömbök C#-ban

Mi az a tömb?

A tömb egy **fix méretű** adatszerkezet, amely **azonos típusú** elemek gyűjteményét tárolja egymás mellett a memóriában. Minden elemnek van egy indexe, amely 0-tól kezdődik.

```
// Példa: 5 elemű int tömb
int[] szamok = { 10, 20, 30, 40, 50 };
//           [0] [1] [2] [3] [4]  <- indexek
```

Jellemzők:

- **Fix méret:** Létrehozás után a méret nem változtatható
- **Azonos típus:** Minden elem ugyanolyan típusú
- **Nulla alapú indexelés:** Az első elem indexe 0
- **Referencia típus:** A tömb változó egy memóriacímre mutat

Egydimenziós tömbök

Tömb deklarálása

```
// Szintaxis: típus[] tömbNév;
int[] szamok;           // Tömb deklarálása (még nincs létrehozva)
string[] nevek;
double[] atlagok;
bool[] flags;

// A deklarálás után a tömb értéke null
Console.WriteLine(szamok == null); // true (ha még nem hoztuk létre)
```

Tömb létrehozása

```
// 1. Létrehozás új kulcsszóval (megadva a méretet)
int[] szamok = new int[5]; // 5 elemű tömb, alapértelmezett értékekkel (0)

// 2. Létrehozás és inicializálás egyben
int[] szamok2 = new int[] { 10, 20, 30, 40, 50 };

// 3. Rövidített inicializálás (típus következtetés)
int[] szamok3 = { 10, 20, 30, 40, 50 }; // Leggyakoribb módszer

// 4. Üres tömb
int[] uresTomb = new int[0]; // 0 elemű tömb
```

```
int[] uresTomb2 = { };          // Ugyanaz

// Különböző típusok
string[] nevek = { "Anna", "Béla", "Cecil" };
double[] atlagok = { 4.5, 3.8, 5.0 };
bool[] flags = { true, false, true };
char[] betuk = { 'A', 'B', 'C' };
```

Alapértelmezett értékek

Amikor tömböt hozunk létre **new** kulcsszóval, az elemek alapértelmezett értéket kapnak:

```
int[] szamok = new int[3];      // { 0, 0, 0 }
double[] lebegok = new double[3]; // { 0.0, 0.0, 0.0 }
bool[] logikai = new bool[3];   // { false, false, false }
string[] szovegek = new string[3]; // { null, null, null }

Console.WriteLine(szamok[0]);    // 0
Console.WriteLine(szovegek[0]);  // (üres - null)

// Ellenőrzés
if (szovegek[0] == null)
{
    Console.WriteLine("Az elem null");
}
```

Tömb elemek elérése és módosítása

Indexelés

```
int[] szamok = { 10, 20, 30, 40, 50 };

// Elem lekérése indexszel (0-tól kezdődik)
int első = szamok[0];    // 10
int második = szamok[1]; // 20
int utolsó = szamok[4];  // 50

// Utolsó elem lekérése
int utolsó2 = szamok[szamok.Length - 1]; // 50

Console.WriteLine($"Első elem: {első}");
Console.WriteLine($"Utolsó elem: {utolsó}");

// Elem módosítása
szamok[0] = 100; // Első elem megváltoztatása
szamok[2] = 300; // Harmadik elem megváltoztatása

Console.WriteLine(szamok[0]); // 100
```

```
Console.WriteLine(szamok[2]); // 300

// Index túllépés - HIBA!
// int hiba = szamok[10]; // IndexOutOfRangeException!

// Biztonságos hozzáférés
int index = 2;
if (index >= 0 && index < szamok.Length)
{
    Console.WriteLine(szamok[index]);
}
```

Gyakorlati példák

```
// Hőmérsékletek tárolása
double[] homersekletek = new double[7]; // Hét nap
homersekletek[0] = 18.5; // Hétfő
homersekletek[1] = 20.0; // Kedd
homersekletek[2] = 19.5; // Szerda
homersekletek[3] = 21.0; // Csütörtök
homersekletek[4] = 22.5; // Péntek
homersekletek[5] = 23.0; // Szombat
homersekletek[6] = 21.5; // Vasárnap

// Osztályzatok
int[] osztalyzatok = { 5, 4, 3, 5, 4 };

// Termékek árak
decimal[] arak = { 1500.50m, 2300.00m, 890.99m };

// Felhasználók
string[] felhasznalok = { "admin", "user1", "user2" };

// Kijelölések (checkbox-ok)
bool[] kivalasztva = { true, false, true, false };
```

Tömb bejárása

for ciklus (indexszel)

```
int[] szamok = { 10, 20, 30, 40, 50 };

// Tömb bejárása
for (int i = 0; i < szamok.Length; i++)
{
    Console.WriteLine($"Index {i}: {szamok[i]}");
}

/* Kimenet:
```

```
Index 0: 10
Index 1: 20
Index 2: 30
Index 3: 40
Index 4: 50
*/

// Visszafelé bejárás
for (int i = szamok.Length - 1; i >= 0; i--)
{
    Console.WriteLine(szamok[i]);
}
// Kimenet: 50, 40, 30, 20, 10

// Páros indexű elemek
for (int i = 0; i < szamok.Length; i += 2)
{
    Console.WriteLine(szamok[i]);
}
// Kimenet: 10, 30, 50
```

foreach ciklus (csak olvasásra)

```
int[] szamok = { 10, 20, 30, 40, 50 };

// Elemek bejárása foreach-el (nincs index)
foreach (int szam in szamok)
{
    Console.WriteLine(szam);
}

// foreach-nél NEM LEHET módosítani az elemeket
string[] nevek = { "anna", "béla", "cecil" };

foreach (string nev in nevek)
{
    // nev = nev.ToUpper(); // HIBA! Nem módosítható
    Console.WriteLine(nev.ToUpper()); // OK - kiírás módosítva
}

// Ha módosítani kell, használj for ciklust
for (int i = 0; i < nevek.Length; i++)
{
    nevek[i] = nevek[i].ToUpper(); // OK
}
```

while és do-while ciklusok

```
int[] szamok = { 10, 20, 30, 40, 50 };

// while ciklus
int index = 0;
while (index < szamok.Length)
{
    Console.WriteLine(szamok[index]);
    index++;
}

// do-while ciklus
int i = 0;
do
{
    Console.WriteLine(szamok[i]);
    i++;
} while (i < szamok.Length);
```

Tömb tulajdonságok és műveletek

Length tulajdonság

```
int[] szamok = { 10, 20, 30, 40, 50 };

// Tömb hossza (elemek száma)
int hossz = szamok.Length; // 5
Console.WriteLine($"A tömb {hossz} elemű.");

// Felhasználás ciklusokban
for (int i = 0; i < szamok.Length; i++)
{
    Console.WriteLine(szamok[i]);
}

// Üres tömb ellenőrzés
int[] ures = { };
if (ures.Length == 0)
{
    Console.WriteLine("A tömb üres.");
}

// Null ellenőrzés fontos!
int[] nullTomb = null;
// Console.WriteLine(nullTomb.Length); // NullReferenceException!

// Biztonságos ellenőrzés
if (nullTomb != null && nullTomb.Length > 0)
{
    // ...
}
```

```
    Console.WriteLine("Van elem a tömbben.");  
}
```

Gyakorlati példák tömb bejárásra

```
// Összeg számítás  
int[] szamok = { 10, 20, 30, 40, 50 };  
int osszeg = 0;  
  
for (int i = 0; i < szamok.Length; i++)  
{  
    osszeg += szamok[i];  
}  
Console.WriteLine($"Összeg: {osszeg}"); // 150  
  
// foreach-el  
int osszeg2 = 0;  
foreach (int szam in szamok)  
{  
    osszeg2 += szam;  
}  
  
// Átlag számítás  
double atlag = (double)osszeg / szamok.Length;  
Console.WriteLine($"Átlag: {atlag}"); // 30  
  
// Maximum keresés  
int[] ertekek = { 34, 12, 78, 23, 90, 45 };  
int max = ertekek[0];  
  
for (int i = 1; i < ertekek.Length; i++)  
{  
    if (ertekek[i] > max)  
    {  
        max = ertekek[i];  
    }  
}  
Console.WriteLine($"Maximum: {max}"); // 90  
  
// Minimum keresés  
int min = ertekek[0];  
foreach (int ertek in ertekek)  
{  
    if (ertek < min)  
    {  
        min = ertek;  
    }  
}  
Console.WriteLine($"Minimum: {min}"); // 12  
  
// Páros számok számlálása
```

```
int parosSzam = 0;
foreach (int szam in szamok)
{
    if (szam % 2 == 0)
    {
        parosSzam++;
    }
}
Console.WriteLine($"Páros számok: {parosSzam}");

// Elem keresése
int keresett = 30;
int talalat = -1;

for (int i = 0; i < szamok.Length; i++)
{
    if (szamok[i] == keresett)
    {
        talalat = i;
        break;
    }
}

if (talalat >= 0)
{
    Console.WriteLine($"Megtalálva a {talalat}. indexen");
}
else
{
    Console.WriteLine("Nem található");
}
```

Array osztály metódusai

A `System.Array` osztály hasznos metódusokat biztosít tömbök kezelésére.

Sort - rendezés

```
// Növekvő rendezés
int[] szamok = { 50, 10, 30, 20, 40 };
Array.Sort(szamok);
// szamok most: { 10, 20, 30, 40, 50 }

foreach (int szam in szamok)
{
    Console.WriteLine(szam);
}

// String rendezés (ABC sorrend)
string[] nevek = { "Cecil", "Anna", "Béla" };
```

```
Array.Sort(nevek);  
// nevek most: { "Anna", "Béla", "Cecil" }  
  
// Csökkenő rendezés  
Array.Sort(szamok);  
Array.Reverse(szamok);  
// szamok most: { 50, 40, 30, 20, 10 }
```

Reverse - megfordítás

```
int[] szamok = { 1, 2, 3, 4, 5 };  
Array.Reverse(szamok);  
// szamok most: { 5, 4, 3, 2, 1 }  
  
foreach (int szam in szamok)  
{  
    Console.WriteLine(szam);  
}
```

IndexOf és LastIndexOf - elem keresése

```
int[] szamok = { 10, 20, 30, 20, 40 };  
  
// IndexOf - első előfordulás indexe  
int index1 = Array.IndexOf(szamok, 20); // 1  
int index2 = Array.IndexOf(szamok, 100); // -1 (nem található)  
  
// LastIndexOf - utolsó előfordulás  
int index3 = Array.LastIndexOf(szamok, 20); // 3  
  
Console.WriteLine($"20 első helye: {index1}");  
Console.WriteLine($"20 utolsó helye: {index3}");  
  
// Gyakorlati használat  
string[] gyumolcsok = { "alma", "körte", "barack", "alma" };  
int almaIndex = Array.IndexOf(gyumolcsok, "alma");  
  
if (almaIndex >= 0)  
{  
    Console.WriteLine($"Alma megtalálva: {almaIndex}. index");  
}
```

Clear - elemek törlése (alapértékre állítás)

```
int[] szamok = { 10, 20, 30, 40, 50 };
```



```
// Az összes elem 0-ra állítása
Array.Clear(szamok, 0, szamok.Length);
// szamok most: { 0, 0, 0, 0, 0 }

// Csak bizonyos elemek törlése
int[] szamok2 = { 10, 20, 30, 40, 50 };
Array.Clear(szamok2, 1, 3); // Index 1-től 3 elemet töröl
// szamok2 most: { 10, 0, 0, 0, 50 }

string[] nevek = { "Anna", "Béla", "Cecil" };
Array.Clear(nevek, 0, nevek.Length);
// nevek most: { null, null, null }
```

Copy - másolás

```
int[] eredeti = { 10, 20, 30, 40, 50 };

// Teljes másolás
int[] masolat = new int[5];
Array.Copy(eredeti, masolat, eredeti.Length);
// masolat: { 10, 20, 30, 40, 50 }

// Részleges másolás
int[] reszleges = new int[3];
Array.Copy(eredeti, 1, reszleges, 0, 3);
// reszleges: { 20, 30, 40 }

// FIGYELEM: Egyszerű másolás referencia típusoknál
int[] tomb1 = { 1, 2, 3 };
int[] tomb2 = tomb1; // Csak a referenciát másoljuk!
tomb2[0] = 100;
Console.WriteLine(tomb1[0]); // 100 (mindkettő ugyanarra mutat!)

// Helyes másolás
int[] tomb3 = new int[tomb1.Length];
Array.Copy(tomb1, tomb3, tomb1.Length);
tomb3[0] = 999;
Console.WriteLine(tomb1[0]); // 100 (nem változott)
```

Resize - méret változtatás

```
int[] szamok = { 10, 20, 30 };
Console.WriteLine($"Eredeti méret: {szamok.Length}");

// Méret növelés
Array.Resize(ref szamok, 5);
// szamok most: { 10, 20, 30, 0, 0 }
Console.WriteLine($"Új méret: {szamok.Length}");
```

```
// Méret csökkentés
Array.Resize(ref számok, 2);
// számok most: { 10, 20 }

// FONTOS: Új tömb jön létre a háttérben!
```

Egyéb hasznos metódusok

```
int[] számok = { 10, 20, 30, 40, 50 };

// Exists - van-e olyan elem, ami megfelel a feltételnek
bool vanNagyobb30 = Array.Exists(számok, x => x > 30);
Console.WriteLine(vanNagyobb30); // true

// Find - első olyan elem, ami megfelel
int első = Array.Find(számok, x => x > 25);
Console.WriteLine(első); // 30

// FindAll - minden olyan elem
int[] nagyobbak = Array.FindAll(számok, x => x > 25);
// nagyobbak: { 30, 40, 50 }

// TrueForAll - minden elem megfelel-e
bool mindenPozitiv = Array.TrueForAll(számok, x => x > 0);
Console.WriteLine(mindenPozitiv); // true
```

Többszimenziós tömbök

Kétszimenziós tömb (Mátrix)

Sorokban és oszlopokban tárolja az adatokat.

```
// Deklarálás és inicializálás
int[,] matrix = new int[3, 4]; // 3 sor, 4 oszlop

// Inicializálás értékekkel
int[,] számok =
{
    { 1, 2, 3, 4 },
    { 5, 6, 7, 8 },
    { 9, 10, 11, 12 }
};

// Elem elérése: [sor, oszlop]
int elem = számok[0, 0]; // 1 (első sor, első oszlop)
int elem2 = számok[1, 2]; // 7 (második sor, harmadik oszlop)
int elem3 = számok[2, 3]; // 12 (harmadik sor, negyedik oszlop)
```

```
// Elem módosítása
szamok[0, 0] = 100;

// Méretek lekérése
int sorok = szamok.GetLength(0);    // 3 (sorok száma)
int oszlopok = szamok.GetLength(1);  // 4 (oszlopok száma)

Console.WriteLine($"Sorok: {sorok}, Oszlopok: {oszlopok}");

// Teljes méret (összes elem)
int osszesElem = szamok.Length;    // 12
```

2D tömb bejárása

```
int[,] matrix =
{
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};

// Beágyazott for ciklusokkal
for (int i = 0; i < matrix.GetLength(0); i++) // Sorok
{
    for (int j = 0; j < matrix.GetLength(1); j++) // Oszlopok
    {
        Console.Write($"{matrix[i, j],4}");
    }
    Console.WriteLine();
}
/* Kimenet:
1   2   3
4   5   6
7   8   9
*/

// foreach-el (csak olvasásra)
foreach (int szam in matrix)
{
    Console.Write($"{szam} ");
}
// Kimenet: 1 2 3 4 5 6 7 8 9
```

Gyakorlati 2D példák

```
// Ülésrend (3 sor, 5 hely)
string[,] terem = new string[3, 5];
terem[0, 0] = "Anna";
terem[0, 1] = "Béla";
```

```
terem[1, 2] = "Cecil";
terem[2, 4] = "Dóra";

// Sakktábla
string[,] sakktabla = new string[8, 8];
sakktabla[0, 0] = "Bástya";
sakktabla[0, 1] = "Huszár";
// ... stb.

// Pontszámok (tanulók x tárgyak)
int[,] pontok =
{
    { 85, 90, 78 }, // Tanuló 1: Matek, Fizika, Kémia
    { 92, 88, 95 }, // Tanuló 2
    { 78, 85, 80 }  // Tanuló 3
};

// Átlagok számítása tárgyanként
for (int tárgy = 0; tárgy < pontok.GetLength(1); tárgy++)
{
    int osszeg = 0;
    for (int tanulo = 0; tanulo < pontok.GetLength(0); tanulo++)
    {
        osszeg += pontok[tanulo, tárgy];
    }
    double atlag = (double)osszeg / pontok.GetLength(0);
    Console.WriteLine($"Tárgy {tárgy + 1} átlag: {atlag:F2}");
}

// Amőba tábla (3x3)
char[,] amoba =
{
    { ' ', ' ', ' ' },
    { ' ', ' ', ' ' },
    { ' ', ' ', ' ' }
};

// Játék lépések
amoba[0, 0] = 'X';
amoba[1, 1] = 'O';
amoba[0, 1] = 'X';

// Tábla kirajzolása
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        Console.Write($" {amoba[i, j]} ");
        if (j < 2) Console.Write("|");
    }
    Console.WriteLine();
    if (i < 2) Console.WriteLine("----+----+----");
}
```

Háromdimenziós tömb

```
// 3D tömb: [mélység, sor, oszlop]
int[, ,] kocka = new int[2, 3, 4]; // 2x3x4-es "doboz"

// Inicializálás
int[, ,] tomb3D =
{
    {
        { 1, 2 },
        { 3, 4 }
    },
    {
        { 5, 6 },
        { 7, 8 }
    }
};

// Elem elérése
int elem = tomb3D[0, 1, 1]; // 4

// Méretek
int melyseg = tomb3D.GetLength(0); // 2
int sorok = tomb3D.GetLength(1); // 2
int oszlopok = tomb3D.GetLength(2); // 2

// Bejárás
for (int i = 0; i < tomb3D.GetLength(0); i++)
{
    for (int j = 0; j < tomb3D.GetLength(1); j++)
    {
        for (int k = 0; k < tomb3D.GetLength(2); k++)
        {
            Console.WriteLine($"[{i},{j},{k}] = {tomb3D[i, j, k]}");
        }
    }
}

// Gyakorlati példa: Rubik kocka (3x3x3)
char[, ,] rubik = new char[3, 3, 3];
```

Jagged Array (Szakadt tömbök)

A jagged array tulajdonképpen "**tömb a tömbben**" - minden sor más hosszú lehet.

Jagged array létrehozása

```
// Deklarálás
int[][] jagged = new int[3][]; // 3 soros, de még nincs oszlop

// Sorok inicializálása (különböző hosszúságúak lehetnek!)
jagged[0] = new int[] { 1, 2, 3 }; // 3 elem
jagged[1] = new int[] { 4, 5 }; // 2 elem
jagged[2] = new int[] { 6, 7, 8, 9 }; // 4 elem

// Inicializálás egyben
int[][] szamok =
{
    new int[] { 1, 2, 3 },
    new int[] { 4, 5 },
    new int[] { 6, 7, 8, 9 }
};

// Elem elérése: [sor][oszlop]
int elem1 = szamok[0][0]; // 1
int elem2 = szamok[1][1]; // 5
int elem3 = szamok[2][3]; // 9

// Sor hossza
int elsőSorHossza = szamok[0].Length; // 3
int másodikSorHossza = szamok[1].Length; // 2
```

Jagged array vs 2D tömb

```
// 2D tömb - szabályos mátrix (minden sor ugyanolyan hosszú)
int[,] matrix = new int[3, 4]; // 3x4-es mátrix
// matrix[0] -> 4 elem
// matrix[1] -> 4 elem
// matrix[2] -> 4 elem

// Jagged array - sorok különböző hosszúak lehetnek
int[][] jagged =
{
    new int[] { 1, 2, 3 }, // 3 elem
    new int[] { 4, 5 }, // 2 elem
    new int[] { 6, 7, 8, 9 } // 4 elem
};
```

Jagged array bejárása

```
int[][] jagged =
{
    new int[] { 1, 2, 3 },
    new int[] { 4, 5 },
    new int[] { 6, 7, 8, 9 }
};
```

```

};

// for ciklussal
for (int i = 0; i < jagged.Length; i++) // Sorok
{
    for (int j = 0; j < jagged[i].Length; j++) // Oszlopok (sor hossza!)
    {
        Console.WriteLine($"{jagged[i][j]} ");
    }
    Console.WriteLine();
}

/* Kimenet:
1 2 3
4 5
6 7 8 9
*/

// foreach-el
foreach (int[] sor in jagged)
{
    foreach (int szam in sor)
    {
        Console.WriteLine($"{szam} ");
    }
    Console.WriteLine();
}

```

Gyakorlati jagged array példák

```

// Iskolai osztályok (különböző létszámú osztályok)
string[][] osztalyok =
{
    new string[] { "Anna", "Béla", "Cecil" }, // 1.a - 3 fő
    new string[] { "Dávid", "Emma" }, // 1.b - 2 fő
    new string[] { "Ferenc", "Gabi", "Henrik", "Irén" } // 1.c - 4 fő
};

// Havi költsékek (minden hónap más számú tétel)
decimal[][] koltesek =
{
    new decimal[] { 15000, 8000, 12000 }, // Január - 3 tétel
    new decimal[] { 20000, 5000, 9000, 3000 }, // Február - 4 tétel
    new decimal[] { 7000, 14000 } // Március - 2 tétel
};

// Összeg havonta
for (int honap = 0; honap < koltesek.Length; honap++)
{
    decimal osszeg = 0;
    foreach (decimal koltes in koltesek[honap])
    {

```

```
        osszeg += koltes;
    }
    Console.WriteLine($"{honap + 1}. hónap: {osszeg:N0} Ft");
}

// Pontozási rendszer (játékosok, különböző számú kör)
int[][] pontok =
{
    new int[] { 10, 15, 20, 18 }, // Játékos 1 - 4 kör
    new int[] { 12, 14, 16 },     // Játékos 2 - 3 kör
    new int[] { 8, 11, 13, 15, 17 } // Játékos 3 - 5 kör
};
```

Tömb vs Lista

```
// TÖMB - fix méret
int[] tomb = new int[5]; // 5 elem, nem változtatható
// tomb[5] = 60; // HIBA! Nincs 6. elem

// Ha mégis változtatni akarunk:
Array.Resize(ref tomb, 6); // Új tömb jön létre!

// LISTA - dinamikus méret (később tanulod részletesen)
// List<int> lista = new List<int>();
// lista.Add(10); // Hozzáad
// lista.Add(20);
// lista.Remove(10); // Eltávolít
// Méret automatikusan változik!

// Mikor tömb?
// - Fix elemszám
// - Gyors elérés kell
// - Egyszerű adatszerkezet

// Mikor lista?
// - Változó elemszám
// - Gyakori hozzáadás/törlés
// - Rugalmasság kell
```

Összetett gyakorlati példák

1. Átlagszámítás és értékelés

```
Console.WriteLine("=== TANULÓI PONTSZÁMOK ===\n");

string[] nevek = { "Anna", "Béla", "Cecil", "Dóra", "Erik" };
int[] pontszamok = { 85, 92, 78, 95, 88 };
```



```
// Átlag számítás
int osszeg = 0;
foreach (int pont in pontszamok)
{
    osszeg += pont;
}
double atlag = (double)osszeg / pontszamok.Length;

// Maximum és minimum
int max = pontszamok[0];
int min = pontszamok[0];
int maxIndex = 0;
int minIndex = 0;

for (int i = 1; i < pontszamok.Length; i++)
{
    if (pontszamok[i] > max)
    {
        max = pontszamok[i];
        maxIndex = i;
    }
    if (pontszamok[i] < min)
    {
        min = pontszamok[i];
        minIndex = i;
    }
}

// Eredmények
Console.WriteLine($"Átlag: {atlag:F2}");
Console.WriteLine($"Legjobb: {nevek[maxIndex]} ({max} pont)");
Console.WriteLine($"Legrosszabb: {nevek[minIndex]} ({min} pont)");

Console.WriteLine("\nRészletes eredmények:");
for (int i = 0; i < nevek.Length; i++)
{
    string ertekeles = pontszamok[i] >= atlag ? "Átlag feletti" : "Átlag alatti";
    Console.WriteLine($"{nevek[i]}: {pontszamok[i]} pont - {ertekeles}");
}
```

2. Lottószámok generálása

```
Console.WriteLine("=== LOTTÓ SZÁMOK ===\n");

int[] lotto = new int[5];
Random rnd = new Random();

// Egyedi számok generálása
for (int i = 0; i < lotto.Length; i++)
{
```

```
int ujSzam;
bool letezik;

do
{
    ujSzam = rnd.Next(1, 91); // 1-90 között
    letezik = false;

    // Ellenőrizzük, hogy már van-e ilyen szám
    for (int j = 0; j < i; j++)
    {
        if (lotto[j] == ujSzam)
        {
            letezik = true;
            break;
        }
    }
} while (letezik);

lotto[i] = ujSzam;
}

// Rendezés
Array.Sort(lotto);

// Kiírás
Console.Write("Húzott számok: ");
foreach (int szam in lotto)
{
    Console.Write($"{szam} ");
}
Console.WriteLine();
```

3. Szavazatszámoló rendszer

```
Console.WriteLine("=== SZAVAZÁS EREDMÉNYE ===\n");

string[] jeloltek = { "Jelölt A", "Jelölt B", "Jelölt C", "Jelölt D" };
int[] szavazatok = new int[4];

// Szimuláljuk a szavazást
Random rnd = new Random();
int osszSzavazat = 100;

for (int i = 0; i < osszSzavazat; i++)
{
    int valasztas = rnd.Next(0, 4);
    szavazatok[valasztas]++;
}

// Eredmények
```

```

Console.WriteLine($"Összes szavazat: {osszSzavazat}\n");

for (int i = 0; i < jeloltek.Length; i++)
{
    double szazalek = (double)szavazatok[i] / osszSzavazat * 100;
    Console.WriteLine($"{jeloltek[i]}: {szavazatok[i]} szavazat
({szazalek:F1}%");
}

// Győztes meghatározása
int maxSzavazat = szavazatok[0];
int gyoztesIndex = 0;

for (int i = 1; i < szavazatok.Length; i++)
{
    if (szavazatok[i] > maxSzavazat)
    {
        maxSzavazat = szavazatok[i];
        gyoztesIndex = i;
    }
}

Console.WriteLine($"
Győztes: {jeloltek[gyoztesIndex]} - {maxSzavazat}
szavazattal!");

```

4. Heti hőmérsékletek elemzése

```

Console.WriteLine("=== HETI IDŐJÁRÁS ===\n");

string[] napok = { "Hétfő", "Kedd", "Szerda", "Csütörtök", "Péntek", "Szombat",
"Vasárnap" };
double[] homersekletek = { 18.5, 20.2, 19.8, 21.5, 22.0, 23.5, 22.8 };

// Átlag hőmérséklet
double osszeg = 0;
foreach (double ho in homersekletek)
{
    osszeg += ho;
}
double atlag = osszeg / homersekletek.Length;

// Legmelegebb és leghidegebb nap
double maxHo = homersekletek[0];
double minHo = homersekletek[0];
int maxNap = 0;
int minNap = 0;

for (int i = 1; i < homersekletek.Length; i++)
{
    if (homersekletek[i] > maxHo)
    {

```

```
        maxHo = homersekletek[i];
        maxNap = i;
    }
    if (homersekletek[i] < minHo)
    {
        minHo = homersekletek[i];
        minNap = i;
    }
}

// Napi jelentés
Console.WriteLine("Napi hőmérsékletek:");
for (int i = 0; i < napok.Length; i++)
{
    string trend = "";
    if (i > 0)
    {
        if (homersekletek[i] > homersekletek[i - 1])
            trend = "↑";
        else if (homersekletek[i] < homersekletek[i - 1])
            trend = "↓";
        else
            trend = "→";
    }

    Console.WriteLine($"{napok[i],-10}: {homersekletek[i]:F1}°C {trend}");
}

Console.WriteLine($"\\nHeti átlag: {atlag:F1}°C");
Console.WriteLine($"Legmelegebb: {napok[maxNap]} ({maxHo:F1}°C)");
Console.WriteLine($"Leghidegebb: {napok[minNap]} ({minHo:F1}°C)");
```

Összefoglalás

Tömb alapok

- **Fix méret:** Létrehozás után nem változtatható
- **Nulla alapú indexelés:** Első elem indexe 0
- **Azonos típus:** Minden elem ugyanolyan típusú
- **Length tulajdonság:** Elemek száma

Tömb típusok

- **Egydimenziós:** `int[] szamok = { 1, 2, 3 };`
- **Többdimenziós:** `int[,] matrix = new int[3, 4];`
- **Jagged (szakadt):** `int[][] jagged = new int[3][];`

Tömb műveletek

- **Bejárás:** `for`, `foreach`

- **Rendezés:** `Array.Sort()`
- **Keresés:** `Array.IndexOf()`, `Array.Find()`
- **Másolás:** `Array.Copy()`
- **Fordítás:** `Array.Reverse()`

Előnyök

- Gyors elem elérés ($O(1)$)
- Memóriahatékony
- Egyszerű használat

Hátrányok

- Fix méret
- Drága beszúrás/törlés közepén
- Nem rugalmas

Tipp: Ha dinamikus méretre van szükséged, használj `List<T>` típust (későbbi fejezet)!

C# Alapok Tananyag - 6. Fejezet